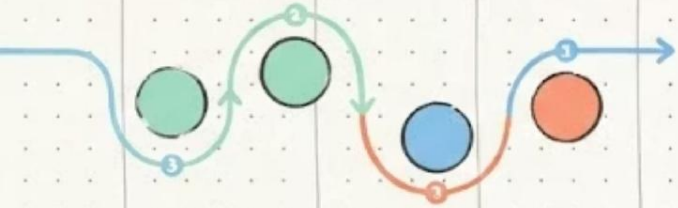


# Gestão do Ciclo de Vida da Aplicação

## Refactoring

Professor Romário da Silva Borges

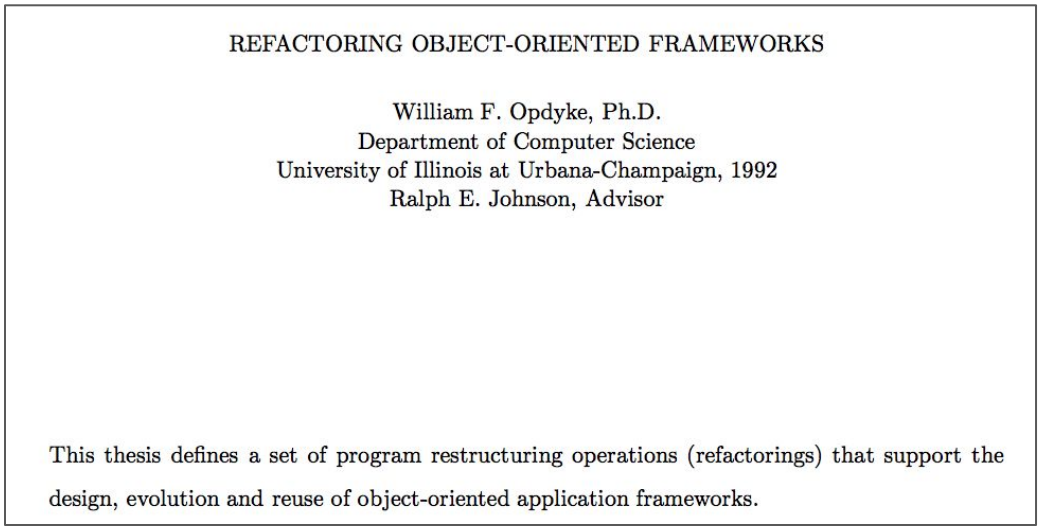


# Manutenção de Software

- Preventiva
- Corretiva
- Adaptativa
- Evolutiva
- Refactoring: melhorias no código ou design

# Refactoring

- Transformações de código que melhoram a manutenibilidade de um sistema mas sem afetar o seu funcionamento externo

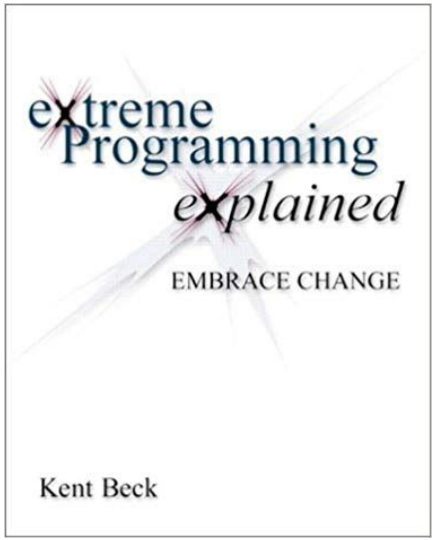


REFACTORING OBJECT-ORIENTED FRAMEWORKS

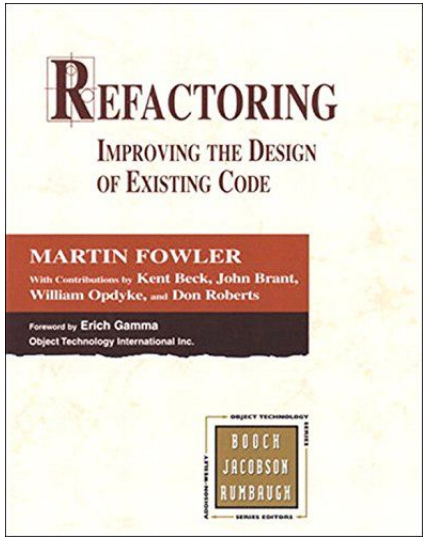
William F. Opdyke, Ph.D.
Department of Computer Science
University of Illinois at Urbana-Champaign, 1992
Ralph E. Johnson, Advisor

This thesis defines a set of program restructuring operations (refactorings) that support the design, evolution and reuse of object-oriented application frameworks.

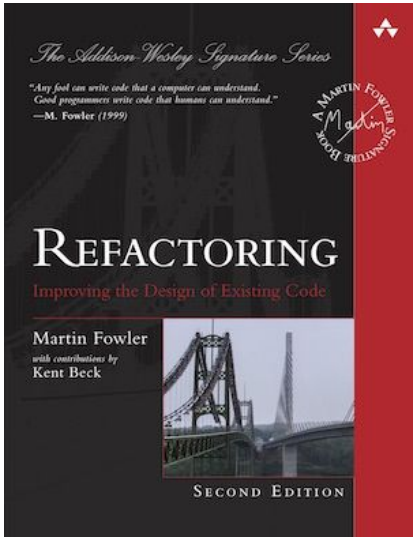
# Conceito tornou-se bastante popular




1999




2000




2018

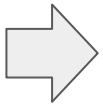
# Catálogo de Refactorings

- Extração de Métodos
- Inline de Métodos
- Movimentação de Métodos
- Extração de Classes
- Renomeação
- etc

# Extração de métodos

# Extração de Métodos

```
void f () {
   ... // A
   ... // B
   ... // C
}
```



```
void g() {  // método extraído
   ... // B
}

void f () {
   ...   // A
   g(); // chama método extraído
   ...   // C
}
```

# Um exemplo real ...

```
void onCreate(SQLiteDatabase database) {// antes da extração
  // cria tabela 1
  database.execSQL("CREATE TABLE " +
           CELL_SIGNAL_TABLE + " (" + COLUMN_ID +
           " INTEGER PRIMARY KEY AUTOINCREMENT, " + ...
  database.execSQL("CREATE INDEX cellID_index ON " + ...);
  database.execSQL("CREATE INDEX cellID_timestamp ON " +...);

  // cria tabela 2
  String SMS_DATABASE_CREATE = "CREATE TABLE " +
           SILENT_SMS_TABLE + " (" + COLUMN_ID +
           " INTEGER PRIMARY KEY AUTOINCREMENT, " + ...
  database.execSQL(SMS_DATABASE_CREATE);
  String ZeroSMS = "INSERT INTO " + SILENT_SMS_TABLE +
           " (Address,Display,Class,ServiceCtr,Message) " +
           "VALUES ('"+ ...
  database.execSQL(ZeroSMS);

  // cria tabela 3
  String LOC_DATABASE_CREATE = "CREATE TABLE " +
           LOCATION_TABLE + " (" + COLUMN_ID +
           " INTEGER PRIMARY KEY AUTOINCREMENT, " + ...
  database.execSQL(LOC_DATABASE_CREATE);
  // mais 200 linhas, criando outras tabelas
}
```

# Antes

```
public void onCreate(SQLiteDatabase database) {
  createCellSignalTable(database);
  createSilentSmsTable(database);
  createLocationTable(database);
  createCellTable(database);
  createOpenCellIDTable(database);
  createDefaultMCCTable(database);
  createEventLogTable(database);
 }
```

## Depois

# Motivações para Extração de Métodos



- Dividir um método grande em métodos menores
- Remover duplicação de código
- Em relação ao método extraído, permitir o seu:
  - Reúso
  - Teste
  - Redefinição em subclasses
  - Chamada recursiva

# *Inline* de métodos (contrário da extração)

```
private void writeContentToFile(final byte[] revision) {
  getVirtualFile().setBinaryContent(revision);
}

private void write(byte[] revision) {
  VirtualFile virtualFile = getVirtualFile();
  ...
  if (document == null) {
       writeContentToFile(revision); // única chamada
  }
   ...
 }
```

# Antes

```
private void writeContentToFile(final byte[] revision) {
  getVirtualFile().setBinaryContent(revision);
}

private void write(byte[] revision) {
  VirtualFile virtualFile = getVirtualFile();
  ...
  if (document == null) {
       writeContentToFile(revision); // única chamada
  }
   ...
 }
```



```
private void write(byte[] revision) {
  VirtualFile virtualFile = getVirtualFile();
  ...
  if (document == null) {
     virtualFile.setBinaryContent(revision);  // após inline
  }
  ...
}
```

**Depois**

# Movimentação de método

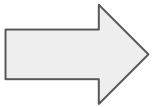
```
-     System.out.println("Average among the N/3 median times: " + PlatformTestUtil.averageAmongMedians(time, 3) + "ms");
+     System.out.println("Average among the N/3 median times: " + ArrayUtil.averageAmongMedians(time, 3) + "ms");

      //System.out.println("JobLauncher.COUNT   = " + JobLauncher.COUNT);
      //System.out.println("JobLauncher.TINY    = " + JobLauncher.TINY_COUNT);
@@ -205,7 +205,7 @@ public int compare(HighlightInfo o1, HighlightInfo o2) {
      }
      FileEditorManagerEx.getInstanceEx(getProject()).closeAllFiles();

-     System.out.println("Average among the N/3 median times: " + PlatformTestUtil.averageAmongMedians(time, 3) + "ms");
+     System.out.println("Average among the N/3 median times: " + ArrayUtil.averageAmongMedians(time, 3) + "ms");
```

# Extração de classes

```
class Pessoa {
  String codigoArea;
  String fone;
  String codigoAreaCelular;
  String celular;
  ...
}
```



```
class Fone { // classe extraída
  String codigoArea;
  String numero;
}

class Pessoa {
  Fone foneFixo;
  Fone foneCelular
  ...
}
```

# Qual o refactoring mais comum?

# Renomeação

(variáveis, parâmetros, métodos, classes, exceções, etc)
(+50% das operações de refactoring)

Dar bons nomes a variáveis é um dos problemas mais difíceis em programação!

# Exercicios

1. Dê o nome de refactorings A e B que se executados em sequência não produzem impacto no código de um sistema.

Ou seja, B reverte as mudanças realizadas por A.

2. Uma mudança no código que foi feita para melhorar o desempenho de um sistema é um refactoring?

# 3. (a) Qual transformação de código foi realizada no programa Java abaixo? (b) Ela é um refactoring? Justifique.

pacote1/A.java

```
package pacote1;
public class A {
  void n() { (new B()).m("abc"); }
```

pacote1/B.java

```
package pacote1;
public class B {
  public void m(Object o) {...}
  void m(String s) {...}
}
```



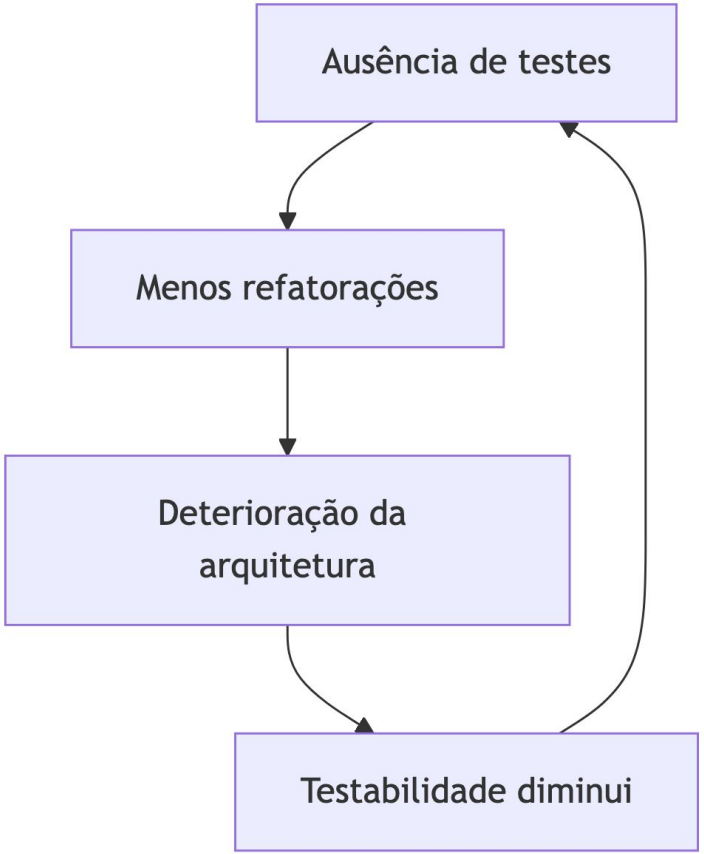
pacote1/A.java

```
package pacote1;
public class A {
  void n() { (new B()).m("abc"); }
```

pacote2/B.java

```
package pacote2;
public class B {
  public void m(Object o) {...}
  void m(String s) {...}
}
```

# Prática de Refactorings




# Falta de Refactorings & Testes (ciclo vicioso)

# Quando refatorar?

1. Refactorings oportunistas
2. Refactorings planejados

# Refactorings Oportunistas

- Realizados no meio de uma outra tarefa de programação
- Tipo de refactoring mais comum

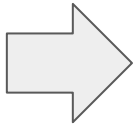
# Refactorings Planejados

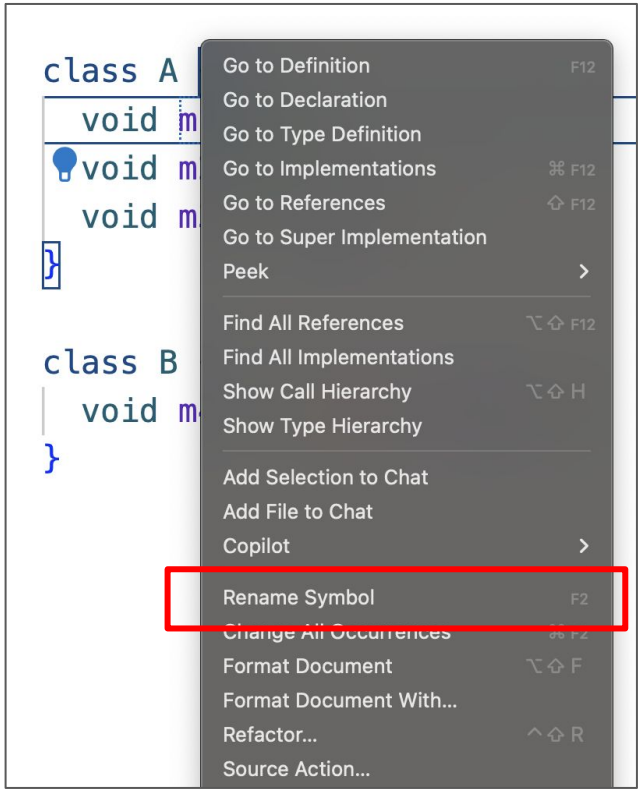
- Correção de um problema de design mais complexo
- Sessão apenas para realização de refactorings

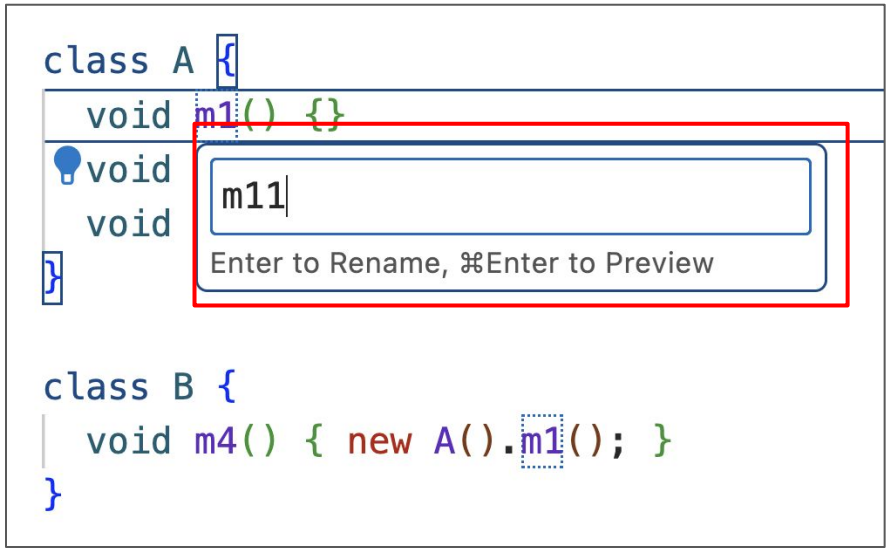
# Refactorings Automatizados

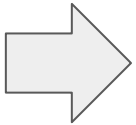
```
class A {
  void m1() {}
  void m2() {}
  void m3() { m1(); }
}

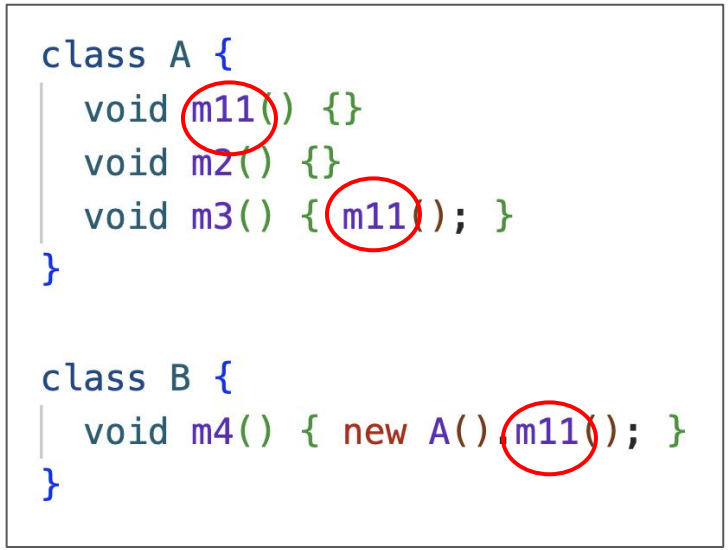
class B {
  void m4() { new A().m1(); }
}
```





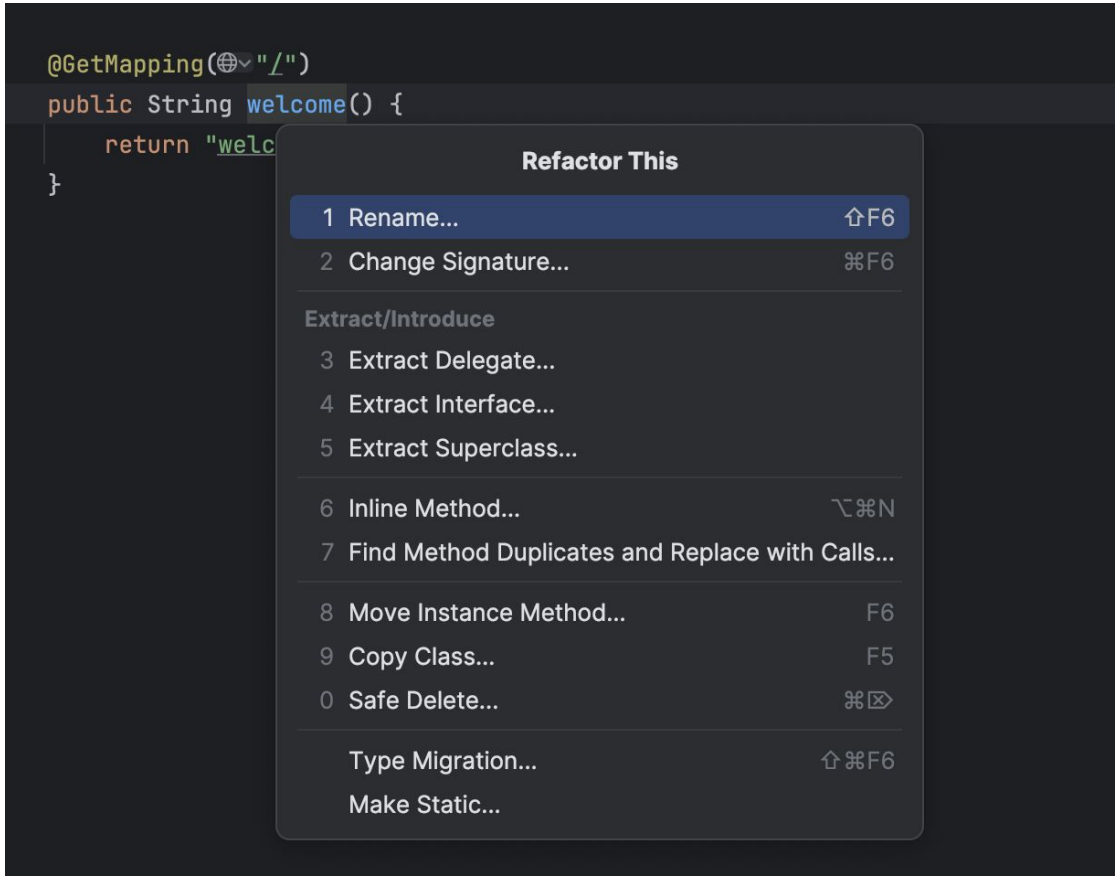

class A {
void m1() {}
void
void
}
m11
Enter to Rename, ⌘Enter to Preview
class B {
void m4() { new A().m1(); }
}




class A {
void m11() {}
void m2() {}
void m3() { m11(); }
}
class B {
void m4() { new A().m11(); }
}

# Outros refactorings do VSCode






# IntelliJ IDEA

https://www.jetbrains.com/help/idea/refactoring-source-code.html

# Mais um refactoring: Remoção de Código Morto

- Código que não está sendo mais usado
- Mais comum do que a gente imagina…

# Exercícios

1. Qual a relação entre a seguinte frase e a prática de refactoring?

> “Para cada mudança que tiver que realizar em um sistema, primeiro torne-a fácil (aviso: isso pode ser difícil), então realize a mudança facilmente.” - Kent Beck

Exercício sobre design de software, testabilidade e refatoração

Primeiro, estude o código a seguir

```
import static javax.swing.JOptionPane.showMessageDialog;

class Dashboard {
  private Stock stock;

  public Dashboard(Stock stock) {
    this.stock = stock;
  }

  public void alert() {
    showMessageDialog(null, "New price " + stock.getName() + " " +
                      stock.getPrice());
  }
}
```



```
class Stock {
  private String name;
  private float price;
  private Dashboard dashboard;

  public Stock(String name, float price) {
    this.name = name;
    this.price = price;
  }

  public void setDashboard(Dashboard dashboard) {
    this.dashboard = dashboard;
  }

  public String getName() {
    return this.name;
  }

  public float getPrice() {
    return this.price;
  }
```

```
// continuação da classe Stock

  public void updatePrice(float price) {
    this.price = price;
    dashboard.alert();
  }

}


public class Main {
  public static void main(String[] args) {
    Stock stock = new Stock("PETR", 40);
    Dashboard dashboard = new Dashboard(stock);
    stock.setDashboard(dashboard);
    stock.updatePrice(50);
  }
}
```

Por que é difícil escrever um teste de unidade para o método updatePrice() de Stock?

Por que é difícil escrever um teste de unidade para o método updatePrice() de Stock?

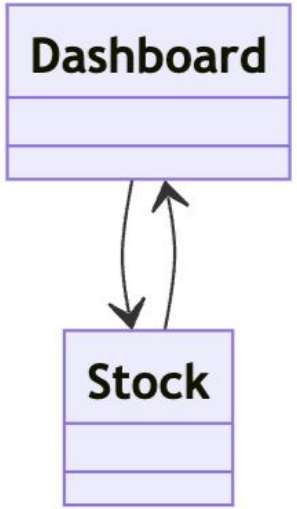
- Dashboard depende de Stock.
- Stock depende de Dashboard.

Problema: Para testar Dashboard, precisamos de Stock, que também depende de Dashboard.

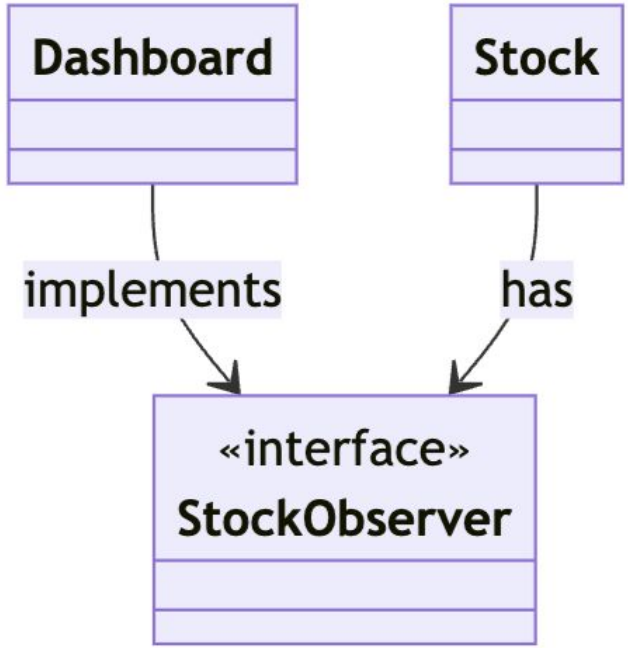
Isso dificulta a criação e o isolamento dos objetos durante os testes.

# Dependências Circulares

- No código anterior, existe uma dependência circular direta entre as classes Dashboard e Stock
- Dependências circulares são indicadores de problemas de design e testabilidade, pois aumentam o acoplamento, dificulta a manutenção e os testes.



# Qual seria uma possível solução?

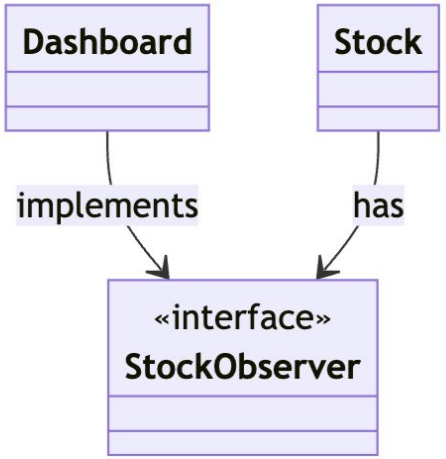

Dashboard
Stock
implements
has
«interface»
StockObserver

```
interface StockObserver {
  public void alert();
}

class Dashboard implements StockObserver {
  ...
  public void alert() {
    showMessageDialog(null, ...);
  }
}

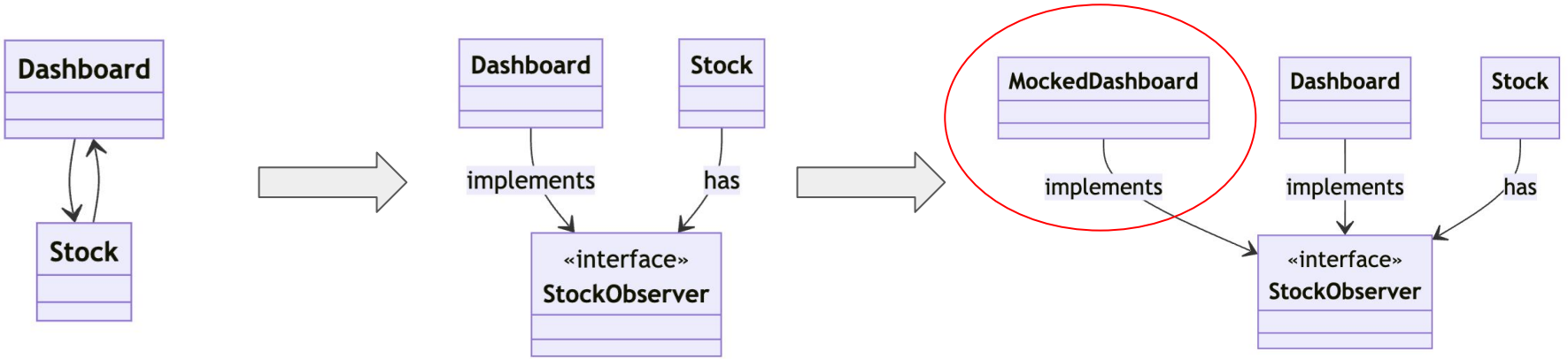
class Stock { ...
  private StockObserver observer;

  public void setObserver(StockObserver observer) {
    this.observer = observer;
  }
  ...
  public void updatePrice(float price) {
    this.price = price;
    observer.alert();
  }
}
```



```
public class Main {
  public static void main(String[] args) {
    Stock stock = new Stock("PETR", 40);
    Dashboard dashboard = new Dashboard(stock);
    stock.setObserver(dashboard);
    stock.updatePrice(50);
  }
}
```

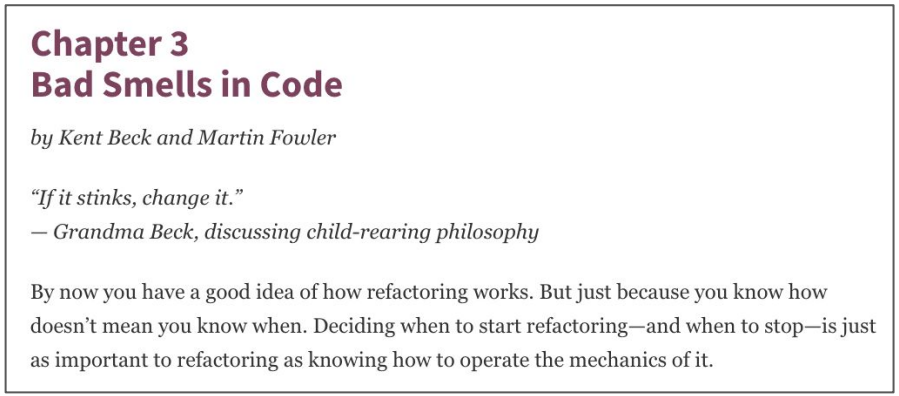
# Resumindo


Dashboard
Stock
Dashboard
Stock
implements
has
«interface»
StockObserver
MockedDashboard
Dashboard
Stock
implements
implements
has
«interface»
StockObserver

# Code smells

# Code (ou Bad) Smells

- Indicadores de código de baixa qualidade
- Difícil de manter, entender, modificar ou testar
- Portanto, candidato a refatoração



## Chapter 3
## Bad Smells in Code

*by Kent Beck and Martin Fowler*

*"If it stinks, change it."*
*— Grandma Beck, discussing child-rearing philosophy*

By now you have a good idea of how refactoring works. But just because you know how doesn't mean you know when. Deciding when to start refactoring—and when to stop—is just as important to refactoring as knowing how to operate the mechanics of it.

# Catálogo de Code Smells

- Código Duplicado
- Métodos Longos
- Classes Grandes
- Feature Envy
- Métodos com Muitos Parâmetros
- Variáveis Globais

- Obsessão por Tipos Primitivos
- Objetos Mutáveis
- Classes de Dados
- Comentários

# Código Duplicado

# Código Duplicado

- Dificulta a manutenção
- Portanto, código candidato a refatoração

# Código Duplicado ⇒ Clones

# Clone Tipo 1 (comentários e espaços)

```
int fatorial(int n) {
  fat = 1;
  for (i = 1; i <= n; i++)
    fat = fat * i;
  return fat;
}
```

Código Original

```
int fatorial(int n) {
  fat=1;
  for (i=1; i<=n; i++)
    fat=fat*i;
  return fat; // retorna fatorial
}
```

# Clone Tipo 2 (tipo 1 + nomes diferentes)

```
int fatorial(int n) {
  fat = 1;
  for (i = 1; i <= n; i++)
    fat = fat * i;
  return fat;
}
```

Código Original

```
int fatorial(int n) {
  f = 1;
  for (j = 1; j <= n; j++)
    f = f * j;
  return f;
}
```

# Clone Tipo 3 (tipo 2 + mudanças em comandos)

```
int fatorial(int n) {
  fat = 1;
  for (i = 1; i <= n; i++)
    fat = fat * i;
  return fat;
}
```

Código Original

```
int fatorial(int n) {
  fat = 1;
  for (j = 1; j <= n; j++)
    fat = fat * j;
  System.out.println(fat); // novo comando
  return fat;
}
```

# Clone Tipo 4 (algoritmos diferentes, mas equivalentes)

```
int fatorial(int n) {
  fat = 1;
  for (i = 1; i <= n; i++)
    fat = fat * i;
  return fat;
}
```

Código Original

```
int fatorial(int n) {
  if (n == 0)
     return 1;
  else return n*fatorial(n-1);
}
```

# Feature Envy

# Feature Envy

- Método que "inveja" dados e métodos de outra classe
- Usa mais métodos e dados dessa outra classe
- Logo, é um candidato a ser movido para ela

```
public class DrawingEditorProxy
             extends AbstractBean implements DrawingEditor {
   ...
   void fireAreaInvalidated2 (AbstractTool abt , Double r ){
     Point p1 = abt.getView().drawingToView (...);
     Point p2 = abt.getView().drawingToView (...);
     Rectangle r=new Rectangle(p1.x,p1.y,p2.x-p1.x p2.y-p1.y);
     abt.fireAreaInvalidated (r);
   }
   ...
}
```

# Variáveis Globais

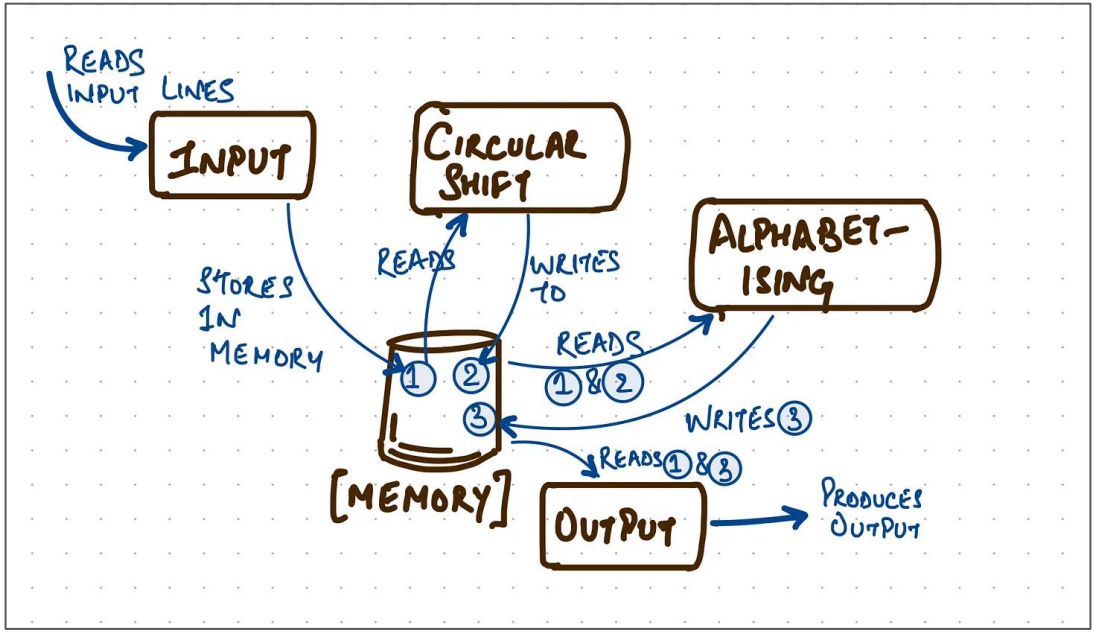
# Variáveis Globais

- Acoplamento ruim
- Dificulta o entendimento de um método

```
void f(...) {
  // computa um determinado valor x
  return x + g; // onde g é uma variável global
}
```

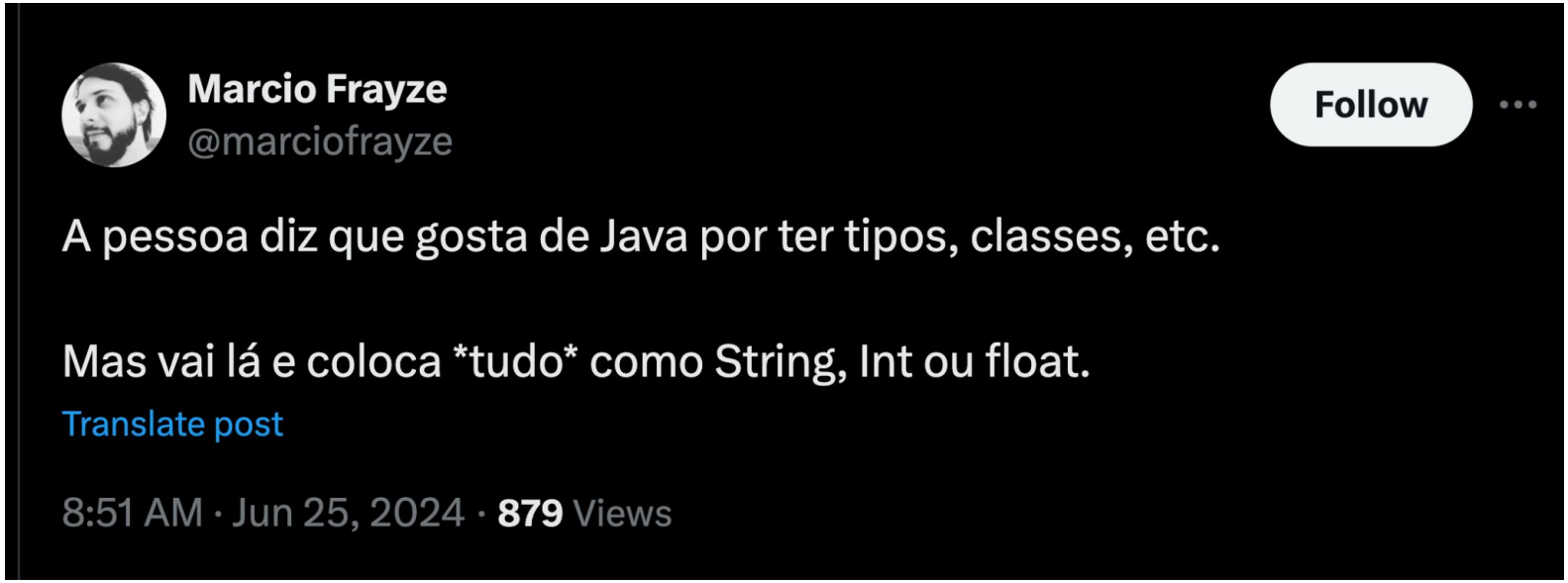
Dependendo de quem chamou f, o resultado pode ser diferente!

# Qual o problema com este código?



# Obsessão por Tipos Primitivos

# Obsessão por Tipos Primitivos em 1 tweet



# Obsessão por Tipos Primitivos

- CEP, Moeda, Data, Hora, Cor, Email, etc **não** devem ser tipos primitivos
- Mas sim de um tipo próprio, com alguns métodos
- Por exemplo, métodos para validação de valores
- Algumas vezes, são chamados de Value Objects.

# Objetos Mutáveis

# Objetos Mutáveis vs Imutáveis

- Mutáveis: estado pode mudar
- Imutáveis: uma vez criados, estado não muda

Exercício: (a) O que será impresso pelo seguinte programa Java? Justifique. (b) Strings em Java são imutáveis ou não?

```
class Main {

  public static void main(String[] args) {
    String s1 = "Hello";
    String s2 = s1.toUpperCase();
    System.out.println(s1); // Hello
    System.out.println(s2); // HELLO
  }

}
```

# Por que objetos imutáveis são "bons"?

- Dão mais segurança a quem criou o objeto
  - Pode-se passar o objeto para outros métodos e ter certeza de que eles não vão alterar o seu estado
- Não estão sujeitos a problemas devido a concorrência
  - Não precisamos de sincronizações, locks, mutex, etc

# Como interpretar esse code smell

- Sempre que possível:
  - Implemente objetos imutáveis
  - Principalmente, objetos simples (CEP, Data, Hora, etc)
- Por outro lado:
  - Em linguagens imperativas é natural ter um bom número de objetos mutáveis

## Python

```
numeros = [1, 2, 3]
numeros[0] = 10
numeros.append(4)
del numeros[1]
print(numeros)


⇒[10, 3, 4]
```

## Python

```
numeros = [1, 2, 3]
numeros[0] = 10
numeros.append(4)
del numeros[1]
print(numeros)


⇒[10, 3, 4]
```

## Elixir

```
numeros = [1, 2, 3]
numeros2 = [10 | tl(numeros)]
numeros3 = numeros2 ++ [4]
numeros4 = List.delete_at(numeros3, 1)
IO.inspect(numeros)
IO.inspect(numeros2)
IO.inspect(numeros3)
IO.inspect(numeros4)

⇒[1, 2, 3]
⇒[10, 2, 3]
⇒[10, 2, 3, 4]
⇒[10, 3, 4]
```

# Comentários

A ideia é a seguinte:

"não comente código ruim, reescreva-o"

-- B. Kerninghan & P. J. Plauger

# Comentários "Ruído" (nada acrescentam …)



# Exemplo #1

```
/** The day of the month. */
    private int dayOfMonth;
```

```
// this function sends an email
void sendEmail() {
  ...
}

// this class holds data for an employee
public class Employee {
  ...
}
```

# Exemplo #2

```
// Add a horizontal scroll bar
hScrollBar = new JScrollBar(JScrollBar.HORIZONTAL);
add(hScrollBar, BorderLayout.SOUTH);

// Add a vertical scroll bar
vScrollBar = new JScrollBar(JScrollBar.VERTICAL);
add(vScrollBar, BorderLayout.EAST);

// Initialize the caret-position related values
caretX    = 0;
caretY    = 0;
caretMemX  = null;
```

Comentários apenas repetem o que já está bem claro no código

Fonte: A Philosophy of Software Design (capítulo 13)

Evidentemente, nem todo comentário é um code smell ...

```
/**
     * Returns a string that is a substring of this string. The
     * substring begins at the specified {@code beginIndex} and
     * extends to the character at index {@code endIndex - 1}.
     * Thus the length of the substring is {@code endIndex-beginIndex}.
     * <p>
     * Examples:
     * <blockquote><pre>
     * "hamburger".substring(4, 8) returns "urge"
     * "smiles".substring(1, 5) returns "mile"
     * </pre></blockquote>
     *
     * @param      beginIndex   the beginning index, inclusive.
     * @param      endIndex     the ending index, exclusive.
     * @return     the specified substring.
     * @throws     IndexOutOfBoundsException  if the
     *             {@code beginIndex} is negative, or
     *             {@code endIndex} is larger than the length of
     *             this {@code String} object, or
     *             {@code beginIndex} is larger than
     *             {@code endIndex}.
     */
    public String substring(int beginIndex, int endIndex) { ... }
```

# Exercícios

1. Seja a seguinte classe `Preco`. Uma vantagem é que ela pode incluir um método (não mostrado) para converter o valor para outras moedas, como dólar. (a) Por que seus objetos são mutáveis? (b) Reimplemente a classe para que seus objetos sejam imutáveis.

```
class Preco {
  ...
  private double valor = 0.0;

  void incrementa(double quant) {
    this.valor+= quant;
  }
  ...
}
```

# Resposta em Java

```
final class Preco {                  // final: proíbe subclasses
  ...
  private final double valor;        // inicializado uma única vez
                                     // (normalmente, na construtora)

  public Preco(double valor) {
    this.valor = valor;
  }

  Preco incrementa(double quant) {
    return new Preco(this.valor + quant);
  }
  ...
}
```

# Resposta em Java, usando records

```
public record Preco(double valor) {

  public Preco incrementa(double quant) {
    return new Preco(valor + quant);
   }

}
```

- Records: recursos para implementação de objetos imutáveis, disponíveis a partir de Java 14
- Sintaxe mais simples e compacta

2. Nos próximos três slides, mostramos o código de uma função de um sistema de código aberto, chamado FitNesse, a qual é usada também em um dos exemplos do livro Clean Code.

(a) Qual code smell existe nesta função?

(b) Qual o principal refactoring que elimina esse smell?

Observação: se preferir, o código da função está aqui.

```
public static String testableHtml(
        PageData pageData,
        boolean includeSuiteSetup
) throws Exception {
    WikiPage wikiPage = pageData.getWikiPage();
    StringBuffer buffer = new StringBuffer();
    if (pageData.hasAttribute("Test")) {
        if (includeSuiteSetup) {
            WikiPage suiteSetup = PageCrawlerImpl.getInheritedPage(
                    SuiteResponder.SUITE_SETUP_NAME, wikiPage
            );
            if (suiteSetup != null) {
                WikiPagePath pagePath =
                                suiteSetup.getPageCrawler().getFullPath(suiteSetup);
                String pagePathName = PathParser.render(pagePath);
                buffer.append("!include -setup .")
                        .append(pagePathName)
                        .append("\n");
            }
        }
```

```
    WikiPage setup = PageCrawlerImpl.getInheritedPage("SetUp", wikiPage);
    if (setup != null) {
        WikiPagePath setupPath = wikiPage.getPageCrawler().getFullPath(setup);
        String setupPathName = PathParser.render(setupPath);
        buffer.append("!include -setup .")
                .append(setupPathName)
                .append("\n");
    }
}
buffer.append(pageData.getContent());
if (pageData.hasAttribute("Test")) {
    WikiPage teardown = PageCrawlerImpl.getInheritedPage("TearDown", wikiPage);
    if (teardown != null) {
        WikiPagePath tearDownPath =
                            wikiPage.getPageCrawler().getFullPath(teardown);
        String tearDownPathName = PathParser.render(tearDownPath);
        buffer.append("!include -teardown .")
                .append(tearDownPathName)
                .append("\n");
    }
```

```
        if (includeSuiteSetup) {
            WikiPage suiteTeardown = PageCrawlerImpl.getInheritedPage(
                    SuiteResponder.SUITE_TEARDOWN_NAME,
                    wikiPage
            );
            if (suiteTeardown != null) {
                WikiPagePath pagePath =
                         suiteTeardown.getPageCrawler().getFullPath(suiteTeardown);
                String pagePathName = PathParser.render(pagePath);
                buffer.append("!include -teardown .")
                        .append(pagePathName)
                        .append("\n");
            }
        }
    }
    pageData.setContent(buffer.toString());
    return pageData.getHtml();

}
```




**Professor Marco Tulio Valente**

Slides disponibilizados pelo Prof. Marco Tulio Valente. Esta apresentação contém adaptações da apresentação original que está disponível na seguinte página: https://engsoftmoderna.info.

**HORSTMANN, C.**

Padrões e Projeto Orientados a Objetos, 2 Edição, Editora Bookman: Porto Alegre, 2007

**BEZERRA, E.**

Princípios de Análise e Projeto de Sistemas com UML, 2a ed., Editora Campus/Elsevier: Rio de Janeiro, 2007

**SOMMERVILLE, I.**

Engenharia de Software. 8a ed., Ed. Pearson: São Paulo, 2008.

**PAULA, Filho W. P.**

Engenharia de Software: Fundamentos, Métodos e Padrões, 3a ed. Ed. LTC: Rio de Janeiro, 2009.

**PRESSMAN, R.**

Software Engineering: A Practitioner's Approach. 5a ed. McGraw Hill : São Paulo, 2000